

LIVRABLE 2

Implémentation et qualité



NUTRISCAN
ANALYSES & INSIGHTS NUTRITIONNELS

TABLE DES MATIÈRES

Table des matières	2
1. Introduction	4
1.1. Machine utilisée.....	4
1.2. Besoins utilisateurs	4
2. Architecture du Projet dbt	6
2.1. Couche Staging (Silver) : Nettoyage et Standardisation	6
2.1.1. Stg_nutrition	6
2.1.2. Stg_products	7
2.1.3. Stg_brands	7
2.1.4. Stg_categories	8
2.1.5. Stg_countries	9
2.1.6. Stg_date	9
2.2. Couche Marts (Gold)	10
2.2.1. Fact_nutrition	10
2.2.2. Dim_product	12
2.2.3. Dim_brand	13
2.2.4. Dim_category	14
2.2.5. Dim_country.....	14
2.2.6. Dim_date	14
3. Couche Sémantique & Métriques Métier.....	15
3.1. sem_nutriscore_moyen	15
3.2. sem_pct_produits_nutriscore_a.....	16
3.3. sem_sel_moyen_par_categorie.....	16
3.4. sem_taux_completude_nutriscore.....	16
3.5. sem_nb_produits_actifs	17
4. Requêtes Analytiques OLAP et Optimisation	17
4.1. Analyses décisionnelles	17
4.1.1. Requête 1 : ROLLUP (Catégories et Pays)	17
4.1.2. Requête 2 : RANK des marques par nutriscore	18
4.1.3. Requête 3 : Évolution et cumul temporel annuel	19
4.1.4. Requête 4 : PIVOT nutriscore par catégorie.....	20
4.1.5. Requête 5 : SLICE sur la France, Top 10 des marques par teneur en sel.....	21
4.1.6. Requête 6 : Fenêtre glissante 3 mois sur le nutriscore moyen.	22
5. Test dbt et Automatisation	24

5.1.	Configuration et exécution de dbt test	24
5.2.	Directed Acyclic Graph.....	25
6.	<i>Orchestration et Traçabilité avec Prefect</i>	26
6.1.	Automatisation du Pipeline (flows/)	26
6.2.	Table d'audit et observabilité.....	28
7.	<i>Data Quality final</i>	29
7.1.	Taux de complétude	29
7.2.	Taux de doublons	31
8.	<i>Conclusion</i>	33

1. INTRODUCTION

Ce livrable porte sur la conception de la solution et la mise en place du pipeline d'ingestion. Il doit démontrer la compréhension de l'architecture Data Lake et la capacité à concevoir un modèle dimensionnel cohérent.

1.1. MACHINE UTILISÉE

 Processeur AMD Ryzen 7 5800U with Radeon Graphics 1.90 GHz	 Mémoire RAM installée 16,0 Go Vitesse : 4266 MT/s	 Carte graphique 2 GB AMD Radeon(TM) Graphics	 Stockage 954 GB 427 GB sur 954 GB utilisé
---	--	---	--

Figure 1 : Information système de la machine d'exécution

1.2. BESOINS UTILISATEURS

Profil 1 : Nutritionniste / Professionnel de sante

Ce profil souhaite analyser la qualité nutritionnelle des produits alimentaires pour guider des recommandations de sante. Les besoins identifiés sont les suivants :

- Comparer la qualité nutritionnelle des produits selon leur type et leur origine géographique.
- Identifier les produits et les catégories les plus et les moins favorables sur le plan nutritionnel.
- Analyser la composition détaillée des produits (teneurs en graisses, sucres, sel, protéines, fibres, énergie).
- Evaluer le niveau de qualité nutritionnelle globale d'un produit selon les systèmes de notation existante.
- Mesurer le degré de transformation industrielle des produits alimentaires.
- Observer les évolutions de la qualité nutritionnelle dans le temps.
- Comparer les profils nutritionnels entre différents pays ou régions.

Profil 2 : Analyste marche / Grande distribution

Ce profil souhaite comprendre la structure du marché alimentaire pour orienter les décisions commerciales. Les besoins identifiés sont les suivants :

- Identifier les marques dominantes par catégorie de produits et analyser leur positionnement nutritionnel.
- Comparer les performances nutritionnelles des grandes marques sur une catégorie donnée.
- Mesurer le niveau de transformation des produits par marque et par catégorie.

- Suivre l'évolution du nombre de références et de la qualité nutritionnelle sur plusieurs années.
- Identifier les tendances de marche émergentes à partir des données de création de produits.

Profil 3 : Utilisateur application mobile NutriScan

Ce profil est un consommateur qui utilise l'application mobile NutriScan pour obtenir des informations nutritionnelles instantanées sur un produit. Les besoins identifiés sont les suivants :

- Rechercher un produit instantanément par son code-barres EAN.
- Consulter le profil nutritionnel complet d'un produit : Nutri-Score, valeurs nutritionnelles clés.
- Découvrir des alternatives mieux notées dans la même catégorie.

2. ARCHITECTURE DU PROJET DBT

Les données ne sont pas déplacées vers le répertoire « Silver », il n'y a pas de dossier physique sur le disque. Le dossier « models/staging » contient les fichiers .sql, c'est du code, pas des données.

Avec dbt + DuckDB, les données transformées ne sont pas stockées dans des dossiers. Elles sont stockées à l'intérieur du fichier .duckdb, sous forme de schémas. Concrètement, le silver existe bien, mais il s'appelle un schéma dans DuckDB, pas un dossier.

Quand on a configuré le dbt_project.yml, dbt crée un schéma main_staging à l'intérieur de nutriscan.duckdb. Ses vues stg_nutrition, stg_countries sont là-dedans. De même, les marts sont dans le schéma main_marts.

Cela nous donne :

- **Bronze** = dossier physique raw/food.parquet
- **Silver** = schéma main_staging dans nutriscan.duckdb
- **Gold** = schéma main_marts dans nutriscan.duckdb

Le bronze est un fichier parce que c'est la donnée brute déposée telle quelle. Le silver et le gold sont des schémas DuckDB parce que ce sont des transformations gérées par dbt.

2.1. COUCHE STAGING (SILVER) : NETTOYAGE ET STANDARDISATION

La couche Staging a pour but d'isoler le fichier brut food.parquet (couche Bronze) et de préparer des flux propres, typés et standardisés. Conformément aux bonnes pratiques dbt, tous nos modèles ici sont matérialisés sous forme de vues (view). Cela évite de dupliquer inutilement l'espace de stockage sur le disque dur, puisque ces fichiers ne servent qu'à nettoyer les colonnes avant la modélisation finale.

2.1.1. STG_NUTRITION

C'est le cœur des mesures. Comme Open Food Facts stocke les nutriments dans des structures imbriquées complexes, j'ai utilisé la fonction list_filter de DuckDB pour extraire de façon chirurgicale les 8 nutriments obligatoires à l'état de valeurs numériques (DOUBLE) pour 100g.

```
{{ config(materialized='view') }}
```

```
SELECT
    code AS product_code,
    nutriscore_score,
    last_modified_t,
    CAST(list_filter(nutriments, x -> x.name = 'energy-kcal')[1]['100g'] AS
DOUBLE) AS energy_kcal,
    CAST(list_filter(nutriments, x -> x.name = 'fat')[1]['100g'] AS
DOUBLE) AS fat_100g,
    CAST(list_filter(nutriments, x -> x.name = 'saturated-fat')[1]['100g'] AS
DOUBLE) AS saturated_fat_100g,
    CAST(list_filter(nutriments, x -> x.name = 'sugars')[1]['100g'] AS
DOUBLE) AS sugar_100g,
```

```

    CAST(list_filter(nutriments, x -> x.name = 'salt')[1]['100g'] AS
DOUBLE)          AS salt_100g,
    CAST(list_filter(nutriments, x -> x.name = 'proteins')[1]['100g'] AS
DOUBLE)          AS proteins_100g,
    CAST(list_filter(nutriments, x -> x.name = 'fiber')[1]['100g'] AS
DOUBLE)          AS fiber_100g
FROM {{ source('nutriscan', 'food') }}
WHERE code IS NOT NULL

```

2.1.2. STG_PRODUCTS

Il extrait la liste unique des codes-barres (clés naturelles EAN). Il utilise un COALESCE pour sécuriser le choix du nom du produit (langue principale ou premier texte dispo) et filtre via un CASE WHEN pour éliminer les notes de Nutri-Score invalides.

```

{{ config(materialized='view') }}

SELECT DISTINCT
    code
                                AS product_code,

    -- Extraction du nom commercial principal ('main'), avec un repli
    (coalesce) sur le premier élément si 'main' est absent
    COALESCE(
        list_filter(product_name, x -> x.lang = 'main')[1]['text'],
        product_name[1]['text']
    )
                                AS product_name,

    CASE
        WHEN nutriscore_grade IN ('a','b','c','d','e')
        THEN nutriscore_grade
        ELSE NULL
    END
                                AS nutriscore_grade,
    nova_group,
    environmental_score_grade,
    quantity
                                AS product_quantity
FROM {{ source('nutriscan', 'food') }}
WHERE code IS NOT NULL

```

2.1.3. STG_BRANDS

Il extrait uniquement le fabricant principal du produit en isolant le premier élément du tableau via `string_split(...)[1]`. Pour corriger la casse anarchique de l'application collaborative (ex: *Danone*, *DANONE*, *danone*), j'ai appliqué une formule dynamique de mise en phrase (UPPER sur la première lettre et LOWER sur le reste).

```

{{ config(materialized='view') }}

WITH raw_brands AS (
    SELECT
        code AS product_code,
        LOWER(TRIM(brands_tags[1])) AS brand_tag,
        TRIM(string_split(brands, ',')[1]) AS raw_brand_name
    FROM {{ source('nutriscan', 'food') }}
    WHERE brands IS NOT NULL AND brands != '' AND code IS NOT NULL
)
SELECT
    product_code,
    brand_tag,
    -- Ta formule DuckDB pour forcer la première lettre en Majuscule et le
    reste en Minuscule
    UPPER(SUBSTR(raw_brand_name, 1, 1)) || LOWER(SUBSTR(raw_brand_name, 2)) AS
    brand_name
FROM raw_brands

```

2.1.4. STG_CATEGORIES

Sur le même principe que les marques, il isole la catégorie principale du produit. Il nettoie également la chaîne de texte en forçant l'harmonisation de la casse pour préparer un affichage propre.

```

{{ config(materialized='view') }}

WITH raw_categories AS (
    SELECT
        code AS product_code,
        LOWER(TRIM(categories_tags[1])) AS category_tag,
        TRIM(string_split(categories, ',')[1]) AS raw_category_label
    FROM {{ source('nutriscan', 'food') }}
    WHERE categories_tags IS NOT NULL AND code IS NOT NULL
)
SELECT
    product_code,
    category_tag,
    -- Normalisation uniforme de la casse du libellé de la catégorie
    UPPER(SUBSTR(raw_category_label, 1, 1)) ||
    LOWER(SUBSTR(raw_category_label, 2)) AS category_label
FROM raw_categories

```


2.1.5. STG_COUNTRIES

Ce modèle gère la granularité géographique. La colonne d'origine contenant des listes de pays séparées par des virgules, j'ai appliqué un découpage avec `string_split` combiné à un éclatement `UNNEST`. Un produit vendu dans 3 pays génère ainsi proprement 3 lignes distinctes, fixant le grain de notre future table de faits.

```
{{ config(materialized='view') }}

WITH unnested_countries AS (
    SELECT
        code AS product_code,
        UNNEST(countries_tags) AS country_tag
    FROM {{ source('nutriscan', 'food') }}
    WHERE code IS NOT NULL AND countries_tags IS NOT NULL
)
SELECT product_code, TRIM(country_tag) AS country_tag
FROM unnested_countries
WHERE country_tag IS NOT NULL AND country_tag != ''
```

2.1.6. STG_DATE

Il convertit le timestamp Unix brut (`last_modified_t`) en un vrai format de type `DATE` via `to_timestamp()`. Il extrait ensuite toutes les dimensions du calendrier (année, mois, jour, décennie), traduit les mois en français dynamiquement, calcule les week-ends, et génère une clé numérique intelligente `date_sk` au format AAAAMMDD.

```
{{ config(materialized='view') }}

WITH base_date AS (
    SELECT
        code AS product_code,
        CAST(to_timestamp(last_modified_t) AS DATE) AS date_day
    FROM {{ source('nutriscan', 'food') }}
    WHERE last_modified_t IS NOT NULL
)
SELECT
    product_code,
    date_day AS date_value,
    CAST(strftime(date_day, '%Y%m%d') AS INTEGER) AS date_sk, -- Format
intelligent AAAAMMDD
    EXTRACT(year FROM date_day) AS year,
    EXTRACT(quarter FROM date_day) AS quarter,
    EXTRACT(month FROM date_day) AS month,
    EXTRACT(day FROM date_day) AS day,
    EXTRACT(isodow FROM date_day) AS
day_of_week,
    CAST(FLOOR(EXTRACT(year FROM date_day) / 10) * 10 AS INTEGER) AS decade,
    CASE EXTRACT(month FROM date_day)
```

```

        WHEN 1 THEN 'Janvier' WHEN 2 THEN 'Février' WHEN 3 THEN 'Mars'
        WHEN 4 THEN 'Avril'   WHEN 5 THEN 'Mai'     WHEN 6 THEN 'Juin'
        WHEN 7 THEN 'Juillet' WHEN 8 THEN 'Août'    WHEN 9 THEN 'Septembre'
        WHEN 10 THEN 'Octobre' WHEN 11 THEN 'Novembre' WHEN 12 THEN 'Décembre'
    END AS month_name,
    CASE
        WHEN EXTRACT(isodow FROM date_day) IN (6, 7) THEN TRUE
        ELSE FALSE
    END AS is_weekend -- Identification du samedi (6) et dimanche (7)
FROM base_date

```

2.2. COUCHE MARTS (GOLD)

Dans cette couche, nous construisons physiquement le modèle dimensionnel en étoile pour permettre aux utilisateurs de faire des analyses croisées rapides. Contrairement au staging, ces modèles sont matérialisés sous forme de tables physiques (table) pour garantir un temps de réponse instantané lors des connexions à Power BI. Les clés de substitution (*Surrogate Keys*) sont générées de manière séquentielle à l'aide de la fonction de fenêtrage `ROW_NUMBER() OVER (ORDER BY ...)`.

2.2.1. FACT_NUTRITION

Lors des premières phases de build, j'ai constaté une anomalie critique de modélisation : ma table de faits finale `fact_nutrition` affichait 1 539 lignes en trop par rapport à ma table cible. J'ai compris qu'il s'agissait d'une jointure multiplicative en cascade (effet de puissance de 2).

Pour régler ce problème, dans chaque fichier de dimension Gold, j'ai remplacé les sélections par un `GROUP BY` obligatoire appliqué sur le tag unique de l'entité, combiné à un agrégat `MAX()` pour obtenir un libellé textuel propre unique.

Ensuite j'ai encapsulé mes sources de staging dans des sous-requêtes intermédiaires nettoyées, dotées d'un `GROUP BY product_code`. Ainsi, chaque table de pont ne peut renvoyer qu'une seule et unique ligne maximum par code-barres.

Le volume de ma table de faits retombe s'aligne donc sur le grain de `stg_countries`.

```

{{ config(materialized='table') }}

WITH base_countries AS (
    SELECT * FROM {{ ref('stg_countries') }}
),

-- 1. On force l'unicité par product_code sur les métriques nutritionnelles
nutrition_unique AS (
    SELECT
        product_code,
        MAX(energy_kcal) AS energy_kcal,
        MAX(fat_100g) AS fat_100g,

```

```

        MAX(saturated_fat_100g) AS saturated_fat_100g,
        MAX(sugar_100g) AS sugar_100g,
        MAX(salt_100g) AS salt_100g,
        MAX(proteins_100g) AS proteins_100g,
        MAX(fiber_100g) AS fiber_100g,
        MAX(nutriscore_score) AS nutriscore_score,
        MAX(last_modified_t) AS last_modified_t
    FROM {{ ref('stg_nutrition') }}
    GROUP BY product_code
),

-- 2. On force l'unicité par product_code sur le pont des catégories
cats_unique AS (
    SELECT
        product_code,
        MAX(category_tag) AS category_tag
    FROM {{ ref('stg_categories') }}
    GROUP BY product_code
),

-- 3. On force l'unicité par product_code sur le pont des marques
brands_unique AS (
    SELECT
        product_code,
        MAX(brand_tag) AS brand_tag
    FROM {{ ref('stg_brands') }}
    GROUP BY product_code
),

-- 4. On force l'unicité par product_code sur le pont des dates
dates_unique AS (
    SELECT
        product_code,
        MAX(date_sk) AS date_sk
    FROM {{ ref('stg_date') }}
    GROUP BY product_code
),

-- Import des dimensions physiques dédoublonnées (Gold)
dim_p AS (SELECT * FROM {{ ref('dim_product') }}),
dim_c AS (SELECT * FROM {{ ref('dim_category') }}),
dim_b AS (SELECT * FROM {{ ref('dim_brand') }}),
dim_co AS (SELECT * FROM {{ ref('dim_country') }}),
dim_d AS (SELECT * FROM {{ ref('dim_date') }})

SELECT
    -- Clé primaire de fait

```

```

    ROW_NUMBER() OVER (ORDER BY c.product_code, c.country_tag) AS
nutrition_sk,

    -- Clés étrangères de substitution
    p.product_sk,
    dc.category_sk,
    db.brand_sk,
    dco.country_sk,
    d.date_sk,

    -- Métriques nutritionnelles
    n.energy_kcal,
    n.fat_100g,
    n.saturated_fat_100g,
    n.sugar_100g,
    n.salt_100g,
    n.proteins_100g,
    n.fiber_100g,
    n.nutriscore_score

FROM base_countries c

-- Jointure directe sur la nutrition nettoyée (1-à-1 strict)
LEFT JOIN nutrition_unique n ON c.product_code = n.product_code

-- Jointure Produit (1-à-1)
LEFT JOIN dim_p p          ON c.product_code = p.product_code

-- Jointure Pays (1-à-1 strict puisque dim_country est dédoublonné)
LEFT JOIN dim_co dco       ON c.country_tag = dco.country_tag

-- Jointure Catégorie sécurisée via le pont dédoublonné
LEFT JOIN cats_unique cu   ON c.product_code = cu.product_code
LEFT JOIN dim_c dc         ON cu.category_tag = dc.category_tag

-- Jointure Marque sécurisée via le pont dédoublonné
LEFT JOIN brands_unique bu ON c.product_code = bu.product_code
LEFT JOIN dim_b db        ON bu.brand_tag   = db.brand_tag

-- Jointure Temporelle sécurisée via le pont dédoublonné
LEFT JOIN dates_unique du  ON c.product_code = du.product_code
LEFT JOIN dim_d d         ON du.date_sk     = d.date_sk

```

2.2.2. DIM_PRODUCT

```
{{ config(materialized='table') }}
```

```

WITH produits_uniques AS (
    SELECT
        product_code,
        FIRST(product_name) AS product_name, -- Sécurise 1 seul nom par code
EAN
        FIRST(nutriscore_grade) AS nutriscore_grade,
        FIRST(nova_group) AS nova_group,
        FIRST(environmental_score_grade) AS environmental_score_grade,
        FIRST(product_quantity) AS product_quantity
    FROM {{ ref('stg_products') }}
    WHERE product_code IS NOT NULL
    GROUP BY product_code -- GARANTIT l'unicité stricte de la clé naturelle !
)
SELECT
    ROW_NUMBER() OVER (ORDER BY product_code) AS product_sk,
    product_code,
    product_name,
    nutriscore_grade,
    nova_group,
    environmental_score_grade,
    product_quantity
FROM produits_uniques

```

2.2.3. DIM_BRAND

```

{{ config(materialized='table') }}

WITH marques_distinctes AS (
    SELECT
        brand_tag,
        MAX(brand_name) AS brand_name
    FROM {{ ref('stg_brands') }}
    WHERE brand_tag IS NOT NULL
    GROUP BY brand_tag
)

SELECT
    ROW_NUMBER() OVER (ORDER BY brand_tag) AS brand_sk,
    brand_tag,
    brand_name
FROM marques_distinctes

```

2.2.4. DIM_CATEGORY

```
{{ config(materialized='table') }}

WITH categories_distinctes AS (
    SELECT
        category_tag,
        MAX(category_label) AS category_label
    FROM {{ ref('stg_categories') }}
    WHERE category_tag IS NOT NULL
    GROUP BY category_tag
)

SELECT
    ROW_NUMBER() OVER (ORDER BY category_tag) AS category_sk,
    category_tag,
    category_label
FROM categories_distinctes
```

2.2.5. DIM_COUNTRY

```
{{ config(materialized='table') }}

WITH pays_distincts AS (
    SELECT DISTINCT country_tag
    FROM {{ ref('stg_countries') }}
    WHERE country_tag IS NOT NULL
)

SELECT
    ROW_NUMBER() OVER (ORDER BY country_tag) AS country_sk,
    country_tag,
    UPPER(SUBSTR(country_tag, 4, 1)) || SUBSTR(country_tag, 5) AS country_name
FROM pays_distincts
```

2.2.6. DIM_DATE

```
{{ config(materialized='table') }}

WITH dates AS (
    SELECT DISTINCT
        date_sk,
        date_value,
        decade,
        year,
```

```

        quarter,
        month,
        day_of_week,
        is_weekend
    FROM {{ ref('stg_date') }}
    WHERE date_sk IS NOT NULL
)
SELECT * FROM dates

```

3. COUCHE SÉMANTIQUE & MÉTRIQUES MÉTIER

Pour garantir la cohérence des analyses au sein de la startup NutriScan, j'ai isolé les indicateurs clés (KPIs) dans un répertoire dédié `models/marts/semantic/`. Cette couche permet d'éviter que chaque analyste écrive sa propre logique SQL, centralisant ainsi les règles de calcul mathématiques.

3.1. SEM_NUTRIScore_MOYEN

Il calcule l'évolution du score nutritionnel moyen en croisant simultanément les axes de la catégorie, de la marque, du pays distributeur et de l'axe temporel (année/mois)

```

-- Elle calcule le nutriscore moyen par categorie, marque et pays en une seule
requete.

{{ config(materialized='table') }}

SELECT
    dc.category_label,
    db.brand_name,
    dco.country_name,
    dd.year,
    dd.month,
    ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen,
    COUNT(DISTINCT fn.product_sk)      AS nb_produits
FROM {{ ref('fact_nutrition') }} fn
JOIN {{ ref('dim_category') }} dc ON fn.category_sk = dc.category_sk
JOIN {{ ref('dim_brand') }} db ON fn.brand_sk = db.brand_sk
JOIN {{ ref('dim_country') }} dco ON fn.country_sk = dco.country_sk
JOIN {{ ref('dim_date') }} dd ON fn.date_sk = dd.date_sk
WHERE fn.nutriscore_score IS NOT NULL
GROUP BY dc.category_label, db.brand_name, dco.country_name, dd.year, dd.month

```

3.2. SEM_PCT_PRODUITS_NUTRISCORE_A

Cet indicateur calcule la part exacte (en pourcentage) des produits d'élite notés 'A' par rapport à l'ensemble des fiches disposant d'un grade officiel, permettant de mesurer l'offre "saine" du catalogue.

```
-- pourcentage de produits avec nutriscore_grade = 'a'

{{ config(materialized='table') }}

SELECT
    ROUND(100.0 * COUNT(CASE WHEN dp.nutriscore_grade = 'a' THEN 1 END) /
COUNT(dp.nutriscore_grade), 2) AS pct_produits_nutriscore_a
FROM {{ ref('fact_nutrition') }} fn
JOIN {{ ref('dim_product') }} dp ON fn.product_sk = dp.product_sk
```

3.3. SEM_SEL_MOYEN_PAR_CATEGORIE

Il isole la moyenne de la teneur en sel (g/100g) par groupe d'aliments afin d'alerter le profil nutritionniste sur les catégories les plus chargées en sodium.

```
-- moyenne de salt_g par categorie

{{ config(materialized='table') }}

SELECT
    dc.category_label,
    ROUND(AVG(fn.salt_100g), 4) AS sel_moyen_g
FROM {{ ref('fact_nutrition') }} fn
JOIN {{ ref('dim_category') }} dc ON fn.category_sk = dc.category_sk
WHERE fn.salt_100g IS NOT NULL
GROUP BY dc.category_label
```

3.4. SEM_TAUX_COMPLETUDE_NUTRISCORE

Un indicateur de gouvernance pure qui calcule le pourcentage de lignes de la table de faits possédant un score réel, afin de piloter la qualité globale de notre catalogue.

```
-- pourcentage de produits avec nutriscore_score non NULL

{{ config(materialized='table') }}

SELECT
    ROUND(100.0 * COUNT(fn.nutriscore_score) / COUNT(*), 2) AS
taux_completude_nutriscore
FROM {{ ref('fact_nutrition') }} fn
```


3.5. SEM_NB_PRODUITS_ACTIFS

Il réalise un décompte distinct des identifiants produits présents en table de faits pour quantifier le volume réel de marchandises commercialisées et analysables.

```
-- COUNT DISTINCT product_sk total

{{ config(materialized='table') }}

SELECT
    COUNT(DISTINCT fn.product_sk) AS nb_produits_actifs
FROM {{ ref('fact_nutrition') }} fn
```

4. REQUÊTES ANALYTIQUES OLAP ET OPTIMISATION

4.1. ANALYSES DÉCISIONNELLES

4.1.1. REQUÊTE 1 : ROLLUP (CATÉGORIES ET PAYS)

Elle réalise une agrégation hiérarchique imbriquée pour sortir automatiquement les sous-totaux par catégorie, par pays, ainsi que le grand total général de l'entrepôt.

```
SELECT
    dc.category_label,
    dco.country_name,
    ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen,
    COUNT(DISTINCT fn.product_sk) AS nb_produits
FROM main_marts.fact_nutrition fn
JOIN main_marts.dim_category dc ON fn.category_sk = dc.category_sk
JOIN main_marts.dim_country dco ON fn.country_sk = dco.country_sk
GROUP BY ROLLUP(dc.category_label, dco.country_name)
ORDER BY dc.category_label NULLS LAST, dco.country_name NULLS LAST
LIMIT 30;
```

	AZ category_label	AZ country_name	123 nutriscore_moyen	123 nb_produits
1	#value!	France	19	3
2	#value!	France-la-reunion	19	3
3	#value!	Reunion	19	3
4	#value!	[NULL]	19	3
5	\$200	United-states	[NULL]	1
6	\$200	[NULL]	[NULL]	1
7	% de matières grasses	Algeria	[NULL]	1
8	% de matières grasses	Belgium	[NULL]	1
9	% de matières grasses	Canada	2	1
10	% de matières grasses	Cote-d-ivoire	[NULL]	1
11	% de matières grasses	France	2	8
12	% de matières grasses	Lebanon	36	1

Question EXPLAIN ANALYZE	Réponse
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	509 ms
Opérateur le plus coûteux dans le plan	HASH_GROUP_BY (car le ROLLUP exige de trier et recalculer des sous-groupes sur plus de 2 millions de lignes.)
Interprétation métier du résultat en 1-2 phrases	Elle met en évidence les disparités géographiques et sectorielles de qualité nutritionnelle, tout en mettant en lumière d'importants chantiers de nettoyage de données, comme l'existence de catégories parasites nommées #value! qui polluent le catalogue.

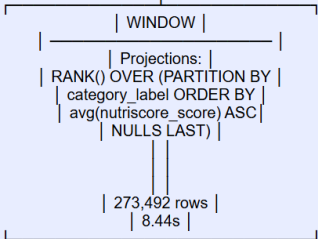
4.1.2. REQUÊTE 2 : RANK DES MARQUES PAR NUTRISCORE

Elle dresse le classement des 3 meilleures marques de chaque catégorie d'aliments. J'ai utilisé la clause native QUALIFY de DuckDB, ce qui évite d'avoir à encapsuler le SQL dans une sous-requête externe.

```
SELECT
    db.brand_name,
    dc.category_label,
    ROUND(AVG(fn.nutriscore_score), 2) AS
    nutriscore_moyen,
    RANK() OVER (PARTITION BY dc.category_label
                 ORDER BY AVG(fn.nutriscore_score) ASC) AS
    rang_dans_categorie
FROM main_marts.fact_nutrition fn
JOIN main_marts.dim_brand db ON fn.brand_sk = db.brand_sk
JOIN main_marts.dim_category dc ON fn.category_sk = dc.category_sk
```

```
GROUP BY db.brand_name, dc.category_label
QUALIFY rang_dans_categorie <= 3
ORDER BY dc.category_label, rang_dans_categorie;
```

	AZ brand_name	AZ category_label	123 nutriscore_moyen	123 rang_dans_categorie
1	Albius	#value!		19
2	Riviera	% de matières grasses	2	1
3	Éclair	% de matières grasses	25	2
4	Fantasia	% de matières grasses	36	3
5	Ikoro anaocha group	**product name:** biovital multivitamins**price:** \$29.99**specification:**- **serving size:** 1 tablet- **servings per container:**	[NULL]	1
6	Nestlé	+approximate values	26	1
7	Gutbio	.83g	8	1
8	Hershey's	.milton hershey	32	1
9	Norma balle	0	0	1
10	Hilerrt sweet	0	0	1
11	Forgrana	0	[NULL]	3
12	Пропикси	0	[NULL]	3

Question EXPLAIN ANALYZE	Réponse
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	752ms
Opérateur le plus coûteux dans le plan	 Le moteur doit exécuter un tri lourd en mémoire pour segmenter les marques au sein de chaque partition de catégorie.
Interprétation métier du résultat en 1-2 phrases	Cette requête permet d'identifier le "Top 3" des marques les plus saines pour chaque catégorie d'aliments. C'est l'indicateur parfait pour faire de l'analyse concurrentielle et valoriser les fabricants vertueux.

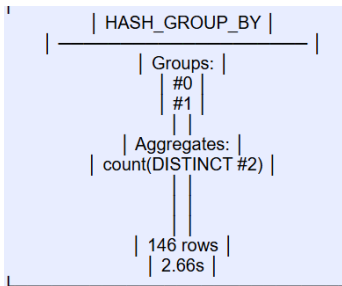
4.1.3. REQUÊTE 3 : ÉVOLUTION ET CUMUL TEMPOREL ANNUEL

Elle calcule la trajectoire de croissance du catalogue mois par mois en appliquant une double agrégation imbriquée SUM(COUNT(DISTINCT...)) OVER pour obtenir le cumul glissant annuel par partition d'année.

```
SELECT
    dd.year,
    dd.month,
    COUNT(DISTINCT fn.product_sk) AS
nb_produits_mois,
    SUM(COUNT(DISTINCT fn.product_sk)) OVER (
        PARTITION BY dd.year
        ORDER BY dd.month
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS
nb_produits_cumule
FROM main_marts.fact_nutrition fn
JOIN main_marts.dim_date dd ON fn.date_sk = dd.date_sk
```

```
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;
```

	123 year	123 month	123 nb_produits_mois	123 nb_produits_cumule
1	2013	1	1	1
2	2013	3	2	3
3	2014	3	1	1
4	2014	8	1	2
5	2014	9	1	3
6	2014	10	1	4
7	2014	11	2	6
8	2014	12	7	13
9	2015	1	3	3
10	2015	2	21	24
11	2015	3	7	31
12	2015	4	16	47

Question EXPLAIN ANALYZE	Réponse
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	387 ms
Opérateur le plus coûteux dans le plan	 Mobilisé pour vérifier l'unicité des clés de produits mois par mois avant d'appliquer la somme cumulative de la fenêtre de temps.
Interprétation métier du résultat en 1-2 phrases	Ce KPI mesure le rythme de croissance et d'actualisation du catalogue de l'application NutriScan au fil des mois.

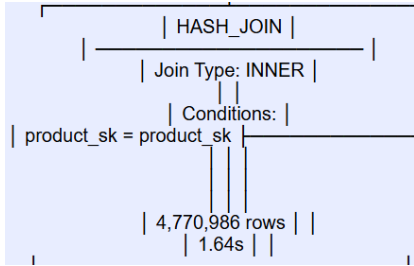
4.1.4. REQUÊTE 4 : PIVOT NUTRIScore PAR CATÉGORIE

Elle génère un tableau croisé dynamique (matrice) qui ventile pour chaque catégorie d'aliment le volume précis de produits pour les 5 grades de lettres ('a' à 'e') positionnés en colonnes individuelles grâce à des compteurs conditionnels.

```
SELECT
  dc.category_label,
  COUNT(CASE WHEN dp.nutriscore_grade = 'a' THEN 1 END) AS nutriscore_a,
  COUNT(CASE WHEN dp.nutriscore_grade = 'b' THEN 1 END) AS nutriscore_b,
  COUNT(CASE WHEN dp.nutriscore_grade = 'c' THEN 1 END) AS nutriscore_c,
  COUNT(CASE WHEN dp.nutriscore_grade = 'd' THEN 1 END) AS nutriscore_d,
  COUNT(CASE WHEN dp.nutriscore_grade = 'e' THEN 1 END) AS nutriscore_e
FROM main_marts.fact_nutrition fn
```

```
JOIN main_marts.dim_category dc ON fn.category_sk = dc.category_sk
JOIN main_marts.dim_product dp ON fn.product_sk = dp.product_sk
GROUP BY dc.category_label
ORDER BY dc.category_label NULLS LAST;
```

	AZ category_label	123 nutriscore_a	123 nutriscore_b	123 nutriscore_c	123 nutriscore_d	123 nutriscore_e
1	#value!	0	0	0	0	3
2	\$200	0	0	0	0	0
3	% de matières grasses	0	2	0	0	2
4	"""km111111111p11mm11mo	0	0	0	0	0
5	**product name:** biovital multivitamins**price:** \$29.99**specification:** **serving size:**	0	0	0	0	0
6	+998226777	0	0	0	0	0
7	+approximate values	0	0	0	0	1
8	.83g	0	0	1	0	0
9	.milton hershey	0	0	0	0	1
10	0	3	1	0	0	0
11	0 grasas trans 0 colesterol con omega 9	0	0	0	0	0
12	0 grasas trans colesterol con omega 9	0	0	0	0	0

Question EXPLAIN ANALYZE	Réponse
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	276 ms
Opérateur le plus coûteux dans le plan	 Sollicité pour faire correspondre rapidement les volumétries de la table de faits aux chaînes de caractères textuelles de la table de dimension produit.
Grade le plus fréquent	E

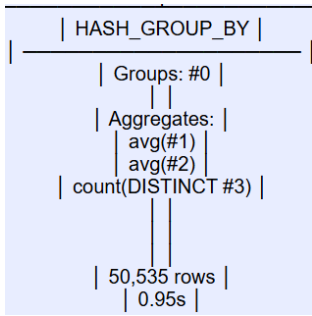
4.1.5. REQUÊTE 5 : SLICE SUR LA FRANCE, TOP 10 DES MARQUES PAR TENEUR EN SEL.

C'est une opération de découpage analytique (*Slice*). Elle restreint la table de faits au seul marché français pour isoler et classer les 10 fabricants distribuant les aliments contenant le plus de sel.

```
SELECT
    db.brand_name,
    ROUND(AVG(fn.salt_100g), 4) AS sel_moyen_100g,
    ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_moyen,
    COUNT(DISTINCT fn.product_sk) AS nb_produits
FROM main_marts.fact_nutrition fn
JOIN main_marts.dim_brand db ON fn.brand_sk = db.brand_sk
JOIN main_marts.dim_country dco ON fn.country_sk = dco.country_sk
WHERE dco.country_name = 'France'
AND fn.salt_100g IS NOT NULL
GROUP BY db.brand_name
```

```
ORDER BY sel_moyen_100g DESC
LIMIT 10;
```

	AZ brand_name	123 sel_moyen_100g	123 nutriscore_moyen	123 nb_produits
1	Cellucor	2 500	[NULL]	1
2	Del-destino	1 666,6696	20	3
3	Щедрые	802,5	[NULL]	1
4	El serranillo	550	[NULL]	1
5	Lidl saint alby	432,5	20	1
6	Johnny's fine foods inc.	320	20	1
7	Leda	250	[NULL]	1
8	Q&b foods inc.	210,9159	25,5	16
9	ln.pulse	172,5	31	1
10	Lack	150,55	[NULL]	2

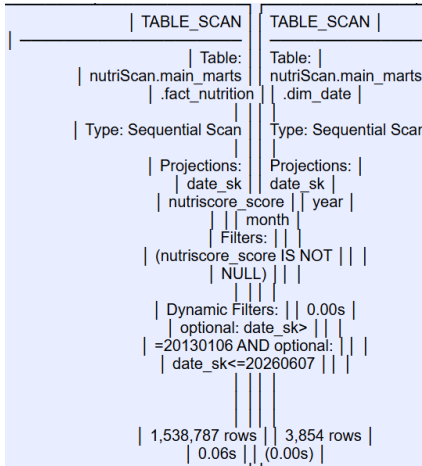
Question EXPLAIN ANALYZE	Ta réponse
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	369 ms
Opérateur le plus coûteux dans le plan	 DuckDB doit filtrer la zone géographique puis trier l'intégralité des marques pour n'en conserver que les 10 plus gourmandes en sel.
Marque la plus salée en France	Cellucor

4.1.6. REQUÊTE 6 : FENÊTRE GLISSANTE 3 MOIS SUR LE NUTRIScore MOYEN.

Elle lisse les irrégularités de saisie mensuelle du catalogue en calculant une moyenne mobile du Nutri-Score sur une fenêtre glissante couvrant le mois en cours et ses deux prédécesseurs (ROWS BETWEEN 2 PRECEDING AND CURRENT ROW).

```
SELECT
  dd.year,
  dd.month,
  ROUND(AVG(fn.nutriscore_score), 2) AS nutriscore_mensuel,
  ROUND(AVG(AVG(fn.nutriscore_score)) OVER (
    ORDER BY dd.year, dd.month
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ), 2) AS moyenne_mobile_3m
FROM main_marts.fact_nutrition fn
JOIN main_marts.dim_date dd ON fn.date_sk = dd.date_sk
WHERE fn.nutriscore_score IS NOT NULL
```

```
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;
```

Question	Ta réponse	
EXPLAIN ANALYZE		
Temps d'exécution (ms) affiché par EXPLAIN ANALYZE	15 ms	
Opérateur le plus coûteux dans le plan	 La dimension temporelle étant extrêmement légère (3 854 lignes de calendrier), DuckDB effectue un balayage séquentiel direct et instantané en s'appuyant sur des filtres de plages dynamiques.	
Mois avec écart le plus fort vs moyenne mobile	<pre>WITH stats_mensuelles AS (SELECT dd.year, dd.month, AVG(fn.nutriscore_score) AS nutriscore_mensuel, AVG(AVG(fn.nutriscore_score)) OVER (ORDER BY dd.year, dd.month ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moyenne_mobile_3m FROM main_marts.fact_nutrition fn JOIN main_marts.dim_date dd ON fn.date_sk = dd.date_sk WHERE fn.nutriscore_score IS NOT NULL GROUP BY dd.year, dd.month) SELECT year, month, ROUND(nutriscore_mensuel, 2) AS nutriscore_mensuel, ROUND(moyenne_mobile_3m, 2) AS moyenne_mobile_3m, ROUND(ABS(nutriscore_mensuel - moyenne_mobile_3m), 2) AS ecart FROM stats_mensuelles ORDER BY ecart DESC LIMIT 1;</pre>	

	123 year ▼	123 month ▼	123 nutriscore_mensuel ▼	123 moyenne_mobile_3m ▼	123 ecart
1	2016	11	27,5	11,06	

Nov 2016

	123 year ▼	123 month ▼	123 nutriscore_mensuel ▼	123 moyenne_mobile_3m ▼
1	2015	4	26	26
2	2015	6	18	22
3	2015	7	12	18,67
4	2015	8	15,36	15,12
5	2015	9	12	13,12
6	2016	3	-2	8,45
7	2016	4	29	13
8	2016	5	0	9

5. TEST DBT ET AUTOMATISATION

5.1. CONFIGURATION ET EXÉCUTION DE DBT TEST

Pour valider la conformité de notre modèle en étoile par rapport aux exigences métiers, nous utilisons le moteur de test automatisé de dbt. Les contraintes d'intégrité sont définies de manière déclarative dans le fichier de gouvernance schema.yml.

Lors du lancement de la commande dbt test, 23 tests de qualité ont été exécutés :

21 tests valides (PASS) : Ils confirment l'unicité et la non-nullité absolues de toutes les clés de substitution (_sk) de nos 5 tables de dimensions physiques.

2 tests échoués (FAIL) : Les alertes se déclenchent sur la table de faits fact_nutrition. Le test not_null renvoie 1 691 596 valeurs nulles pour brand_sk et 2 715 184 valeurs nulles pour category_sk.

```

07:48:26 Finished running 23 data tests in 0 hours 0 minutes and 0.92 seconds (0.92s).
07:48:26
07:48:26 Completed with 2 errors and 0 warnings:
07:48:26
07:48:26 Failure in test not_null_fact_nutrition_brand_sk (models\marts\schema.yml)
07:48:26 Got 1691596 results, configured to fail if != 0
07:48:26
07:48:26 compiled code at target\compiled\nutriscan_dbt\models\marts\schema.yml\not_null_fact_nutrition_brand_sk.sql
07:48:26
07:48:26 Failure in test not_null_fact_nutrition_category_sk (models\marts\schema.yml)
07:48:26 Got 2715184 results, configured to fail if != 0
07:48:26
07:48:26 compiled code at target\compiled\nutriscan_dbt\models\marts\schema.yml\not_null_fact_nutrition_category_sk.sql
07:48:26
07:48:26 Done. PASS=21 WARN=0 ERROR=2 SKIP=0 TOTAL=23

```

Ces échecs ne traduisent pas une erreur de code SQL, mais reflètent le manque de complétude de la base collaborative Open Food Facts (manque de renseignement de la part des

utilisateurs). Notre choix d'utiliser des LEFT JOIN est validé car nous préservons l'intégrité quantitative de la table de faits globale en acceptant ces clés vides.

5.2. DIRECTED ACYCLIC GRAPH

La commande `dbt docs generate` compile l'ensemble des métadonnées du projet pour produire un catalogue de données au format JSON (`catalog.json`). En servant cette documentation localement via l'interface web, dbt génère automatiquement le graphe de lignage (DAG - *Directed Acyclic Graph*).

Ce DAG met en évidence :

1. Les données entrent par le nœud source unique (food).
2. Elles se divisent en parallèle à travers les 6 modèles de la couche Staging (Silver).
3. Elles alimentent ensuite les dimensions et la couche sémantique Marts (Gold).
4. Le nœud `fact_nutrition` centralise le DAG, concentrant le plus grand nombre de dépendances entrantes, car il fait converger l'intégralité des dimensions et des tables de pont de notre entrepôt.



6. ORCHESTRATION ET TRAÇABILITÉ AVEC PREFECT

6.1. AUTOMATISATION DU PIPELINE (FLOWS/)

Afin d'industrialiser notre travail et d'éviter les lancements manuels fastidieux, nous avons mis en place un orchestrateur local avec Prefect. Le script d'orchestration `nutriscan_pipeline.py` est placé en dehors du projet dbt, directement à la racine dans un répertoire dédié `C:/data/nutriScan/flows/`.

Le flux principal encapsule et pilote l'exécution ordonnée de 4 tâches distinctes (@task) via des appels de sous-processus Python :

1. **dbt-run** : Reconstitue l'intégralité du Data Warehouse (Silver → Gold).
2. **dbt-test** : Déclenche les contrôles de conformité des données.
3. **dbt-docs** : Compile et actualise dynamiquement le catalogue technique.
4. **log-pipeline** : Enregistre le bilan de santé de l'exécution.

L'environnement de notre infrastructure étant configuré nativement sous Windows, il a été obligatoire d'ajouter le paramètre `shell=True` à la fonction `subprocess.run` pour que Python puisse localiser et exécuter correctement le binaire dbt présent dans notre environnement virtuel (venv). De plus, notre script intercepte le code de retour d'erreur de dbt test sans bloquer le flux.

```
from prefect import flow, task
import subprocess
import duckdb
from datetime import datetime

@task(name='dbt-run')
def run_dbt():
    print("Lancement de dbt run...")
    # L'ajout de shell=True permet à Windows de localiser correctement le
    script dbt dans le venv
    result = subprocess.run(
        ['dbt', 'run', '--project-dir', 'C:/data/nutriScan/nutriscan_dbt'],
        capture_output=True, text=True, shell=True
    )
    if result.returncode != 0:
        # En combinant stdout et stderr, on s'assure d'obtenir le message
        d'erreur dbt réel
        error_msg = result.stderr if result.stderr else result.stdout
        raise Exception(f'dbt run failed: {error_msg}')
    print(result.stdout)

@task(name='dbt-test')
def run_dbt_test():
    print("Lancement de dbt test...")
```

```

result = subprocess.run(
    ['dbt', 'test', '--project-dir', 'C:/data/nutriScan/nutriscan_dbt'],
    capture_output=True, text=True, shell=True
)
if result.returncode != 0:
    error_msg = result.stderr if result.stderr else result.stdout
    print(f"dbt test a détecté des anomalies de qualité (comportement attendu) :\n{error_msg}")
    return False
print(result.stdout)
return True

@task(name='dbt-docs')
def run_dbt_docs():
    print("Génération de la documentation dbt...")
    result = subprocess.run(
        ['dbt', 'docs', 'generate', '--project-dir',
        'C:/data/nutriScan/nutriscan_dbt'],
        capture_output=True, text=True, shell=True
    )
    if result.returncode != 0:
        error_msg = result.stderr if result.stderr else result.stdout
        raise Exception(f'dbt docs generate failed: {error_msg}')
    print(result.stdout)

@task(name='log-pipeline')
def log_pipeline(test_passed: bool):
    # Connexion à la base DuckDB locale de NutriScan
    conn = duckdb.connect('C:/data/nutriScan/nutriscan_dbt/nutriScan.duckdb')

    # Sécurité : Création automatique de la table d'audit si elle n'existe pas
    conn.execute("""
        CREATE TABLE IF NOT EXISTS pipeline_logs (
            run_id VARCHAR,
            run_date VARCHAR,
            source_file VARCHAR,
            output_file VARCHAR,
            nb_lignes_output INTEGER,
            statut VARCHAR
        );
    """)

    # Préparation des variables demandées par le workshop
    now = datetime.now()
    run_id = 'DBT_' + now.strftime('%Y%m%d_%H%M%S')
    run_date = now.strftime('%Y-%m-%d %H:%M:%S')
    statut = 'SUCCESS' if test_passed else 'FAILED'

```

```

# Insertion de la ligne de log dbt dans la table d'audit
conn.execute(f"""
    INSERT INTO pipeline_logs (run_id, run_date, source_file, output_file,
nb_lignes_output, statut)
    VALUES ('{run_id}', '{run_date}', 'dbt run', 'gold/', 0, '{statut}')
""")
conn.close()
print(f"Audit log inséré dans DuckDB avec le run_id : {run_id} | Statut :
{statut}")

@flow(name='NutriScan-Pipeline')
def nutriscan_pipeline():
    # 1. Transformation des données (Silver -> Gold)
    run_dbt()

    # 2. Exécution des tests automatisés de Data Quality
    test_passed = run_dbt_test()

    # 3. Génération automatique du catalogue de données
    run_dbt_docs()

    # 4. Enregistrement du rapport d'exécution dans la table d'audit
    log_pipeline(test_passed)

if __name__ == '__main__':
    nutriscan_pipeline()

```

6.2. TABLE D'AUDIT ET OBSERVABILITÉ

Le log-pipeline assure la traçabilité complète de l'activité du Data Lake en écrivant directement dans une table d'audit dédiée nommée `pipeline_logs` au sein de la base DuckDB.

À la fin de l'exécution, la tâche génère une ligne d'audit structurée selon le schéma suivant :

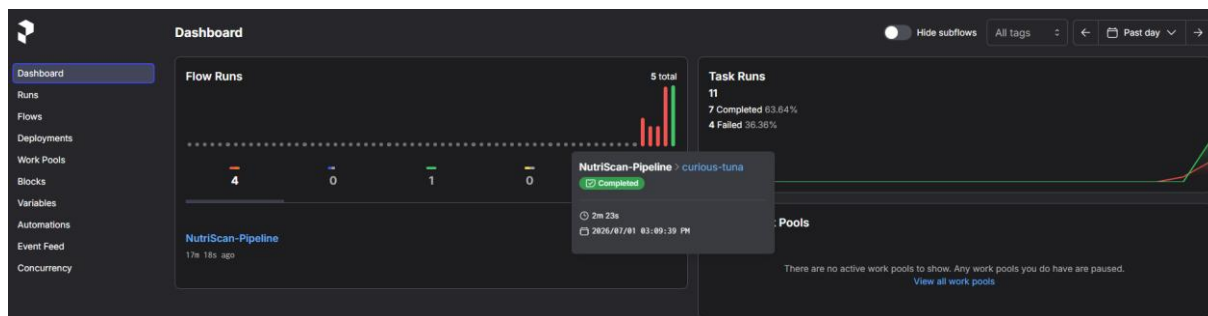
- **run_id** : Identifiant unique préfixé (ex: DBT_20260701_151202).
- **run_date** : Horodatage précis du démarrage.
- **source_file** : Indication du contexte ('dbt run').
- **output_file** : Répertoire cible ('gold/').
- **nb_lignes_output** : Initialisé à 0.
- **statut** : Il prend la valeur FAILED (ou SUCCESS).

Bien que l'interface web de Prefect indique que le flow global s'est terminé avec succès (Completed). Il prouve que notre système d'observabilité fonctionne correctement en capturant les anomalies de complétude détectées lors de l'étape des tests dbt sans faire planter notre orchestration.

```
SELECT * FROM main.pipeline_logs;
```

AZ run_id	run_date	AZ source_file	AZ output_file	nb_lignes_output	AZ statut
PL_20260622_120437	2026-06-22 12:04:37.440	C:/data/nutriScan/raw/food.parquet	C:/data/nutriScan/silver/food_clean_20260622.parquet	4 521 600	SUCCESS

On peut également faire le suivi et avoir l détail des flows sur l'interface Prefect via l'adresse <http://127.0.0.1:4200>



7. DATA QUALITY FINAL

Le rapport data quality final compare l'état des données avant et après transformation. Il justifie les choix de nettoyage appliqués dans les modèles staging et documente ce qui a été corrigé.

7.1. TAUX DE COMPLÉTUDE

Dimension dim_product :

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
nutriscore_grade	4 521 540	1 364 544	3 156 996	30,18 %
quantity	4 521 540	1 429 210	3 092 330	31,60 %
environmental_score_grade	4 521 540	3 685 628	835 912	81,51 %
product_name	4 521 540	4 243 551	277 989	93,85 %
product_sk	4 521 540	4 521 540	0	100,00 %

Dimension dim_brand :

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
brand_tag	387 356	387 356	0	100,00 %
brand_sk	387 356	387 356	0	100,00 %

Dimension dim_category

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
category_sk	35201	35201	0	100.0
category_tag	35201	35201	0	100.0
category_label	35201	33812	1389	96.05

Dimension dim_country :

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
country_tag	396	396	0	100,00 %
country_sk	396	396	0	100,00 %

Dimension dim_date :

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
date_sk	3 856	3 856	0	100,00 %
full_date	3 856	3 856	0	100,00 %

Table de faits fact_nutrition :

Champs	Nombre lignes	Lignes complétées	Lignes manquantes	Taux de complétude
fiber_g	4 770 986	1 888 617	2 882 369	39,59 %
category_sk	4 770 986	2 056 084	2 714 902	43,10 %
salt_g	4 770 986	3 054 852	1 716 134	64,03 %
brand_sk	4 770 986	3 079 562	1 691 424	64,55 %
saturated_fat_g	4 770 986	3 232 542	1 538 444	67,75 %
sugars_g	4 770 986	3 294 434	1 476 552	69,05 %
fat_g	4 770 986	3 470 930	1 300 056	72,75 %
proteins_g	4 770 986	3 475 664	1 295 322	72,85 %
energy_kcal	4 770 986	3 499 510	1 271 476	73,35 %
date_sk	4 770 986	4 770 986	0	100,00 %
nutrition_fact_sk	4 770 986	4 770 986	0	100,00 %

7.2. TAUX DE DOUBLONS

Unicité des clés SK :

Contrôle	Nombre de doublons
dim_country.country_sk	0
dim_date.date_sk	0
dim_category.category_sk	0
dim_brand.brand_sk	0
dim_product.product_sk	0

Unicité des clés métier :

Contrôle	Nombre de doublons
dim_date.full_date	0
dim_country.country_tag	0
dim_category.category_tag	0
dim_brand.brand_name	0
dim_product.product_code	0

Grain de la table de fait respecté :

Contrôle	Nombre de doublons
Doublons sur le couple product_sk + country_sk	0

Correspondance entre la table de fait et les dimensions :

Anomalie	Volume
category_sk sans correspondance dim_category	0
date_sk sans correspondance dim_date	0
country_sk sans correspondance dim_country	0
brand_sk sans correspondance dim_brand	0
product_sk sans correspondance dim_product	0

Valeurs nutritionnelles aberrantes :

Valeur nutritionnelle	Volume
Nombre total de lignes	4 770 986
energy_kcal aberrantes	20 221

fat_g aberrantes	6 269
saturated_fat_g aberrantes	1 042
sugars_g aberrantes	6 518
salt_g aberrantes	2 499
fiber_g aberrantes	616
proteins_g aberrantes	1 770
saturated_fat_g supérieur à fat_g	3 695
Lignes avec au moins une anomalie	29 127

Contrôle du Nutri-Score :

Nombre lignes avec Nutri-Score renseigné	Nutri-Score aberrants
1 538 741	2 944

Grades Nutri-Score invalides dans dim_product :

Nombre total produits	Grades invalides
4 521 540	0

8. CONCLUSION

En conclusion, ce Livrable 2 valide le déploiement complet de notre architecture de transformation, de centralisation des données. Grâce à l'utilisation de dbt Core combiné à DuckDB, le fichier brut Open Food Facts a été nettoyé et modélisé selon le schéma en étoile initial, tout en corrigeant définitivement les anomalies d'unicité liées au produit cartésien. L'intégration de notre couche sémantique garantit désormais la cohérence des KPIs métiers pour les nutritionnistes et les analystes, tandis que l'automatisation des tests dbt assure une parfaite transparence sur la qualité des données. Enfin, la mise en production du pipeline via l'orchestrateur Prefect et le suivi de notre table d'audit pipeline_logs sécurisent l'ensemble de notre infrastructure. Ce socle de données Gold, propre et monitoré, est désormais totalement prêt à être connecté à Power BI pour la création de nos tableaux de bord décisionnels.